

Vyzorix System Architecture Specification

Production Monorepo Pipeline & Native Android Compilation Engine

Author: Senior Software Engineer

Target Runtime: Android OS (ART)

Workspace Engine: pnpm + Turborepo
Monorepo

1. Executive Architectural Blueprint

The Vyzorix application ecosystem uses a strictly separated presentation layer alongside an abstracted, single source-of-truth data core. This strategy ensures seamless code sharing across the desktop web target and the native Android target, entirely managed through a highly integrated mono-repository system.

Rather than duplicating state machines, API routing utilities, types, and logic interfaces, all structural computation is isolated inside specialized shared local packages. The presentation apps act as raw, declarative UI mapping engines.

Workspace Monorepo Core Mappings

```
/ (Workspace Root)
├─ package.json
├─ turbo.json
├─ pnpm-workspace.yaml
├─ packages/
│   └─ api-client/ → Shared TypeScript Type Definitions, HTTP Engines & GraphQL Models
├─ apps/
│   └─ web/ → React 18 + Vite + Tailwind CSS Target
│   └─ mobile/ → Expo SDK / React Native UI Layer (Continuous Native Generation Target)
└─ app.json → Core Android OS Configuration Manifest Mapping
```

2. Separation of Concerns & Shared Monorepo Package Topology

To prevent structural drift and memory usage bloat, code is compartmentalized down to three clean layers:

- **UI & View Layout Layer (Isolated):** Kept strictly independent inside `apps/web/` and `apps/mobile/`. `apps/web/` renders using HTML DOM elements (`<div>`, `<section>`) styled via Tailwind CSS, while `apps/mobile/` utilizes native multi-threaded operational components (`<View>`, `<Text>`) bound to the device's hardware graphics card.
- **State Hooks & Controllers Layer (Shared or Co-located):** View-models and query orchestrators are managed via TanStack Query (React Query) frameworks. These hook up directly to the shared API client modules.
- **Data Logic & Protocol Engine (Unified):** Enclosed fully within `packages/api-client/`. This workspace compiles GraphQL schema operations, Axios instances configured for the Go Gin middleware, and Type mapping declarations before any app target can mount.

3. Structural Workspaces: Monorepo Configuration Mappings

To initialize this cross-dependency compilation structure headlessly in a cloud Linux environment, the core workspace engine parameters must be hardcoded at the monorepo root directory.

3.1 Root Workspace Definition: `pnpm-workspace.yaml`

```
packages:
  - 'apps/*'
  - 'packages/*'
```

3.2 Master Orchestration Pipeline: `turbo.json`

The `turbo.json` configuration handles the build sequence. It maps the dependencies explicitly using the caret syntax (`^`), ensuring packages compile upstream before applications compile downstream.

```
{
  "$schema": "https://turbo.build/schema.json",
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**", "build/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    }
  }
}
```

3.3 Root `package.json` & Scoped Compilation Filtering

By writing targeted filter flags (`--filter`) inside the global scripts, the monorepo engine completely skips compiling the `apps/web/` project when building the mobile application.

```
{
  "name": "vyzorix-monorepo",
  "private": true,
  "packageManager": "pnpm@9.1.0",
  "scripts": {
    "build:web": "turbo run build --filter=web",
    "build:mobile": "turbo run build --filter=mobile",
    "clean": "turbo run clean && rm -rf node_modules"
  },
  "devDependencies": {
    "turbo": "^2.0.0"
  }
}
```

4. Shared Data Logic Framework Configuration

Inside `packages/api-client/package.json`, the package exports its outputs so that both the web and mobile apps can import them cleanly using internal symlinks.

```
{
  "name": "@vyzorix/api-client",
  "version": "1.0.0",
```

```

    "main": "./dist/index.js",
    "types": "./dist/index.d.ts",
    "scripts": {
      "build": "tsup src/index.ts --format cjs,esm --dts"
    },
    "dependencies": {
      "axios": "^1.6.8",
      "graphql": "^16.8.1"
    },
    "devDependencies": {
      "tsup": "^8.0.2",
      "typescript": "^5.4.5"
    }
  }
}

```

5. The Mobile Core Infrastructure (Expo & Metro)

Because the mobile app consumes components from outside its localized path, the standard React Native compiler bundler (Metro) must be reconfigured to prevent path resolution breaks and symlink mapping conflicts.

5.1 Metro Workspace Configuration: `apps/mobile/metro.config.js`

```

const { getDefaultConfig } = require('expo/metro-config');
const path = require('path');

const projectRoot = __dirname;
const workspaceRoot = path.resolve(projectRoot, '../..');

const config = getDefaultConfig(projectRoot);

// Force Metro to track all workspace files in the root monorepo directory
config.watchFolders = [workspaceRoot];

// Force module lookup to correctly fall back to root node_modules paths for pnpm symlinks
config.resolver.nodeModulesPaths = [
  path.resolve(projectRoot, 'node_modules'),
  path.resolve(workspaceRoot, 'node_modules'),
];

// Block nested symbolic lookups from throwing duplication exceptions
config.resolver.disableHierarchicalLookup = true;

module.exports = config;

```

5.2 Expo App Manifest Blueprint: `apps/mobile/app.json`

This file serves as the single declarative blueprint for Continuous Native Generation (CNG). When processed, it auto-generates the entire, complex `android/` folder layout without any manual human coding.

```

{
  "expo": {
    "name": "Vyzorix Mobile",
    "slug": "vyzorix-mobile",
    "version": "1.0.0",

```

```

"orientation": "portrait",
"icon": "./assets/icon.png",
"userInterfaceStyle": "dark",
"splash": {
  "image": "./assets/splash.png",
  "resizeMode": "contain",
  "backgroundColor": "#0f172a"
},
"assetBundlePatterns": ["**/*"],
"android": {
  "package": "dev.vinns.vyzorix",
  "adaptiveIcon": {
    "foregroundImage": "./assets/adaptive-icon.png",
    "backgroundColor": "#0f172a"
  },
  "permissions": [
    "INTERNET",
    "CAMERA"
  ],
  "usesCleartextTraffic": true,
  "proguardRuleFile": "./proguard-rules.pro"
},
"plugins": [
  ["expo-build-properties", {
    "android": {
      "compileSdkVersion": 34,
      "targetSdkVersion": 34,
      "buildToolsVersion": "34.0.0"
    }
  }]
]
}
}

```

Network Communication Strategy for Go Gin Integration

Setting `"usesCleartextTraffic": true` instructs the Android manifest compiler to allow unencrypted HTTP calls during development and staging pipelines inside your cloud sandbox. Without this configuration, the native Android OS kernel will block raw HTTP IP sockets instantly. Turn this flag off for production release builds to enforce strict HTTPS channels.

6. Cloud Orchestration & EAS Build Engine Configurations

To execute builds on Expo Application Services (EAS) remote Linux containers, the operational settings must be mapped out inside `eas.json`.

6.1 Build Profiles: `apps/mobile/eas.json`

```

{
  "cli": {
    "version": ">= 10.0.0",
    "requireWorkspaceRoot": true
  },
  "build": {

```

```

    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal",
      "android": {
        "buildType": "apk"
      }
    },
    "production": {
      "android": {
        "buildType": "app-bundle"
      }
    }
  }
}

```

6.2 Step-by-Step Native Compilation Trigger Flow

To initiate compilation and output a complete native `.apk` or `.aab` package directly from your browser-based headless sandbox workspace, execute the following commands sequentially:

- 1. Install the global deployment engine:** `npm install -g eas-cli`
- 2. Authenticate securely into your remote account:** `eas login`
- 3. Trigger the remote cross-compilation pipeline:**
`eas build --platform android --profile preview --project-dir apps/mobile`

7. The Continuous Native Generation (CNG) Engine Mechanics

When EAS or your sandbox terminal calls `npx expo prebuild --platform android`, an empty directory is turned into a fully populated, production-grade native Android project. The pipeline operates via four synchronized compilation steps:

Pipeline Phase	Under-the-Hood Operation	Resultant Output File System
1. Template Unpacking	Copies a master, unconfigured, native Android app template from <code>expo-template-bare-minimum</code> into the package subfolder.	<code>android/build.gradle</code> <code>android/settings.gradle</code> <code>android/gradlew</code>
2. AST & DOM Mutation	Parses your <code>app.json</code> strings. Rewrites package identifiers, re-structures native Java subdirectories, and appends permission blocks.	<code>android/app/src/main/AndroidManifest.xml</code> <code>java/dev/vinns/vyzorix/MainActivity.java</code>
3. Resource Matrix Generation	Takes single vector assets and cuts them down into five distinct multi-density pixel maps for the device graphics processor.	<code>res/mipmap-xxhdpi/*</code> <code>res/values/strings.xml</code>
4. Build Modification Injection	Executes custom Config Plugins and build properties, appending low-level rules to the app build configurations.	<code>android/app/build.gradle</code> <code>android/app/proguard-rules.pro</code>

8. Complete Production Android File System Specification

Once the generation engine finishes, the directory below is fully populated. This is the complete system specification parsed natively by the Android Runtime (ART) on the device:

```
apps/mobile/android/
├─ build.gradle           # Top-level project build configuration
├─ settings.gradle        # Defines app sub-modules and native dependencies
├─ gradle.properties      # System JVM settings and environment flags
├─ gradlew                # Headless build executable script for Linux environments
├─ gradle/
│   └─ wrapper/
│       └─ gradle-wrapper.properties # Explicit Gradle version engine lock file
└─ app/
    ├─ build.gradle        # App module compiler engine configuration
    ├─ proguard-rules.pro  # Code optimization and anti-reverse-engineering rules
    └─ src/
        └─ main/
            ├─ AndroidManifest.xml # The absolute truth of permissions, names, and activities
            ├─ java/dev/vinns/vyzorix/ # Compiled native entry points
            │   ├─ MainActivity.java # Hooks the UI rendering engine to the OS window
            │   └─ MainApplication.java # Initializes application-wide native modules on boot
            └─ res/           # Hardware resource folders parsed by Android OS
                ├─ drawable/ # Fallback vectors, launch screen graphics
                ├─ mipmap-anydpi-v26/ # Modern adaptive launcher icon XML definitions
                ├─ mipmap-xxhdpi/ # Multi-density raster asset folders (hdpi to xxxhdpi)
                └─ values/
                    ├─ colors.xml # Native system color palettes
                    ├─ strings.xml # App display name and localization records ("Vyzorix
Mobile")
                    └─ styles.xml # System-level window display theme overrides
```

Production Security & Submission Assets

To finalize a Play Store submission, engineers provide a `vyzorix-release.keystore` file. This private cryptographic identity key signs the `.aab` package. EAS can securely manage this inside its cloud key vault, ensuring that you don't have to keep vulnerable signing keys saved inside your public GitHub repository history.